Lab 7: Fuzzing for Input Validation Bugs

Coffee Shop Processor | HEMA SUSHMITHA N | PES1UG24CS667

Objective and method

Use Hypothesis to generate unexpected inputs for drink-code lookup, shot-price averaging and modifier parsing. Complete the starter properties, record failures, fix input validation, and rerun the identical tests. The assigned buggy files are preserved under originals/.

Work completed

1. Created an isolated Python 3.14.6 environment with Hypothesis 6.170.14 and pytest 9.2.0.
2. Ran the buggy interactive script and completed all three starter properties using @given.
3. Added behavior properties, boundary regressions and CLI tests; reproduced failures on the original.
4. Hardened processor.py and reran the same suite; captured all four required stages.

Measurement	Observed result
Buggy application, input moc	KeyError: 'MOC'; exit code 1
Completed suite on original source	34 failed, 10 passed (44 collected tests)
Fixed application, same inputs	Price 5.0; average 1.5; valid modifiers; exit code 0
Identical completed suite on fixed source	44 passed; exit code 0

Testing scope

Eight Hypothesis properties use up to 200 generated examples each, with deterministic generation, shrinking and no saved example database. Thirty-three parametrized regressions and three CLI checks bring the pytest total to 44. Generated-example counts differ from pytest test counts.

Deliverables: fixed processor.py, updated test_processor.py, and this PDF containing all four required screenshots. The failing-test screenshot selects the three starter properties (3 failed, 41 deselected); the complete 34/10 baseline transcript is also saved.

Assigned source | Lab-7 submission

Bugs, counterexamples and fixes

Hypothesis failures are complemented by explicit regressions for known boundary cases.

Area / example	Original behavior	Fix and expected behavior
All functions: generated None	AttributeError on .upper() / .split()	Type checks; lookup/average return None, modifiers return {}
Lookup: generated whitespace; regression moc	KeyError; MOC does not match MOc	Strip and uppercase; correct MOC key; safe lookup
Average: empty CSV, 1,,3, one,two	Unhandled ValueError	Reject invalid CSV as a whole; return None
Average: nan, inf, 1e309, negative prices	Non-finite / invalid monetary values accepted	Require finite nonnegative prices
Average: 1e308,1e308	Intermediate sum overflows to infinity	Scale by maximum; math.fsum; return finite 1e308
Modifiers: generated empty text; malformed segments	Unpacking ValueError or empty keys/values	Skip invalid segments; preserve valid nonempty pairs
Modifiers: note:extra:hot	Too many values to unpack	Split once; preserve extra:hot as value
CLI: invalid input / EOF	Unhandled exception stops processing	Clear validation messages; safe EOF termination

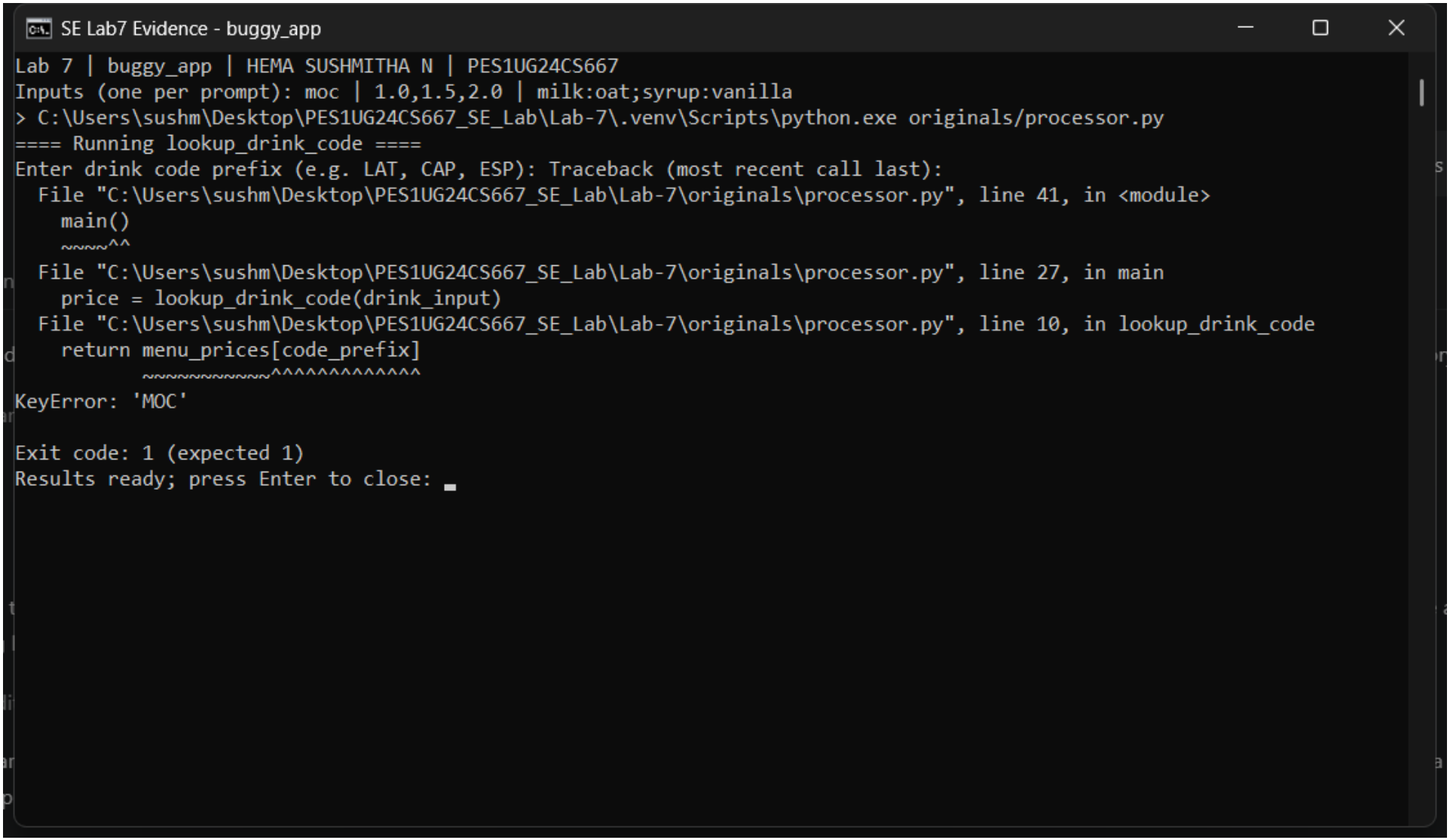
Input policy and conclusions

Unknown/invalid drink and price input returns None. Empty or invalid modifier segments are ignored; duplicate keys use the last value. These policies are explicit design choices where the handout does not prescribe behavior. Valid menu prices and ordinary calculations remain supported.

Hypothesis minimized the three starter failures to None. The original empty CSV raises ValueError during float conversion, rather than an observed divide-by-zero. Passing the suite demonstrates improved robustness for the tested properties, not proof for every input size or resource limit.

Step 2: Buggy Application

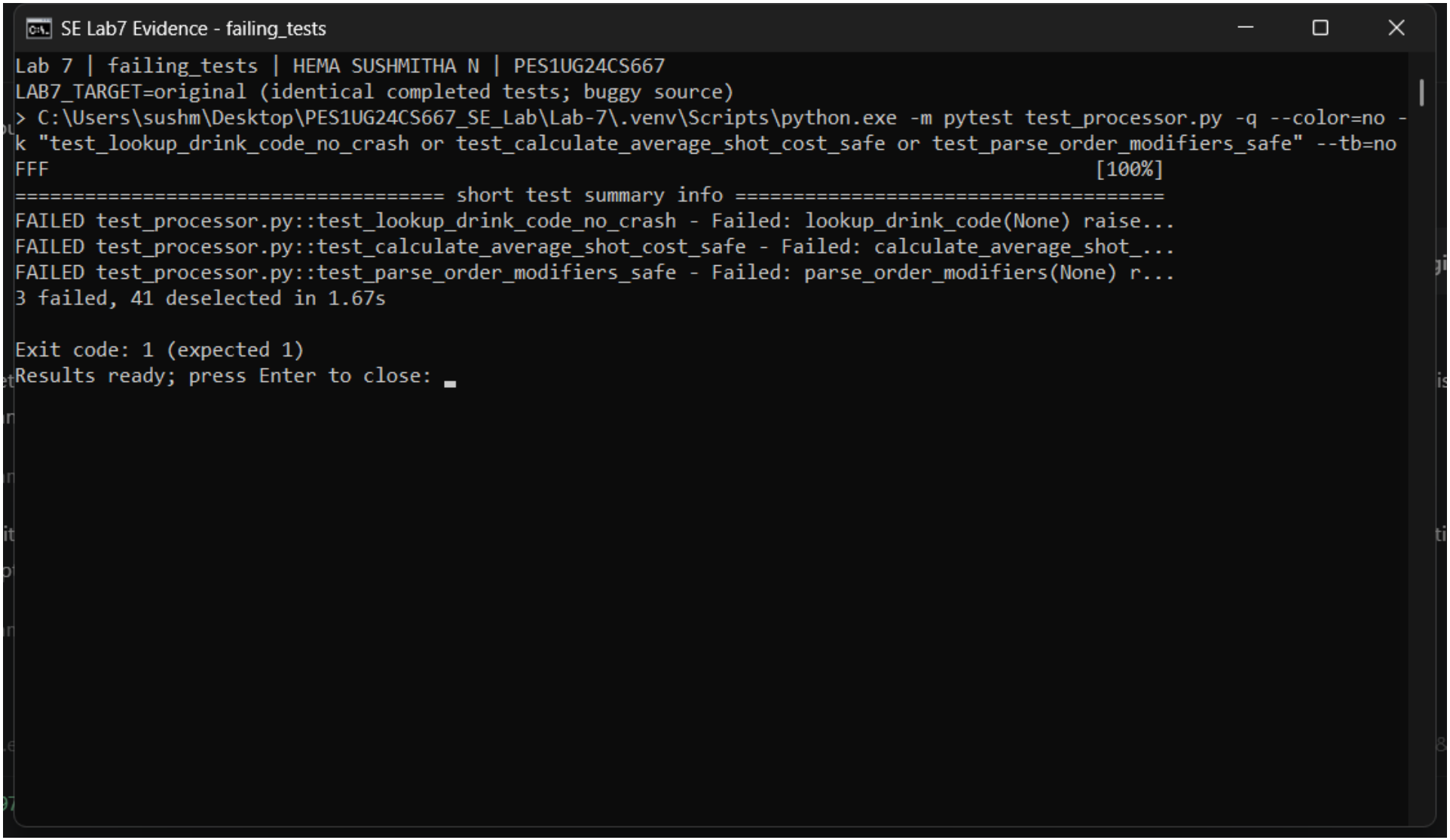
Same demo inputs: moc | 1.0,1.5,2.0 | milk:oat;syrup:vanilla



The original application crashes at the drink lookup with KeyError: 'MOC'.

Step 4: Failing Hypothesis Tests

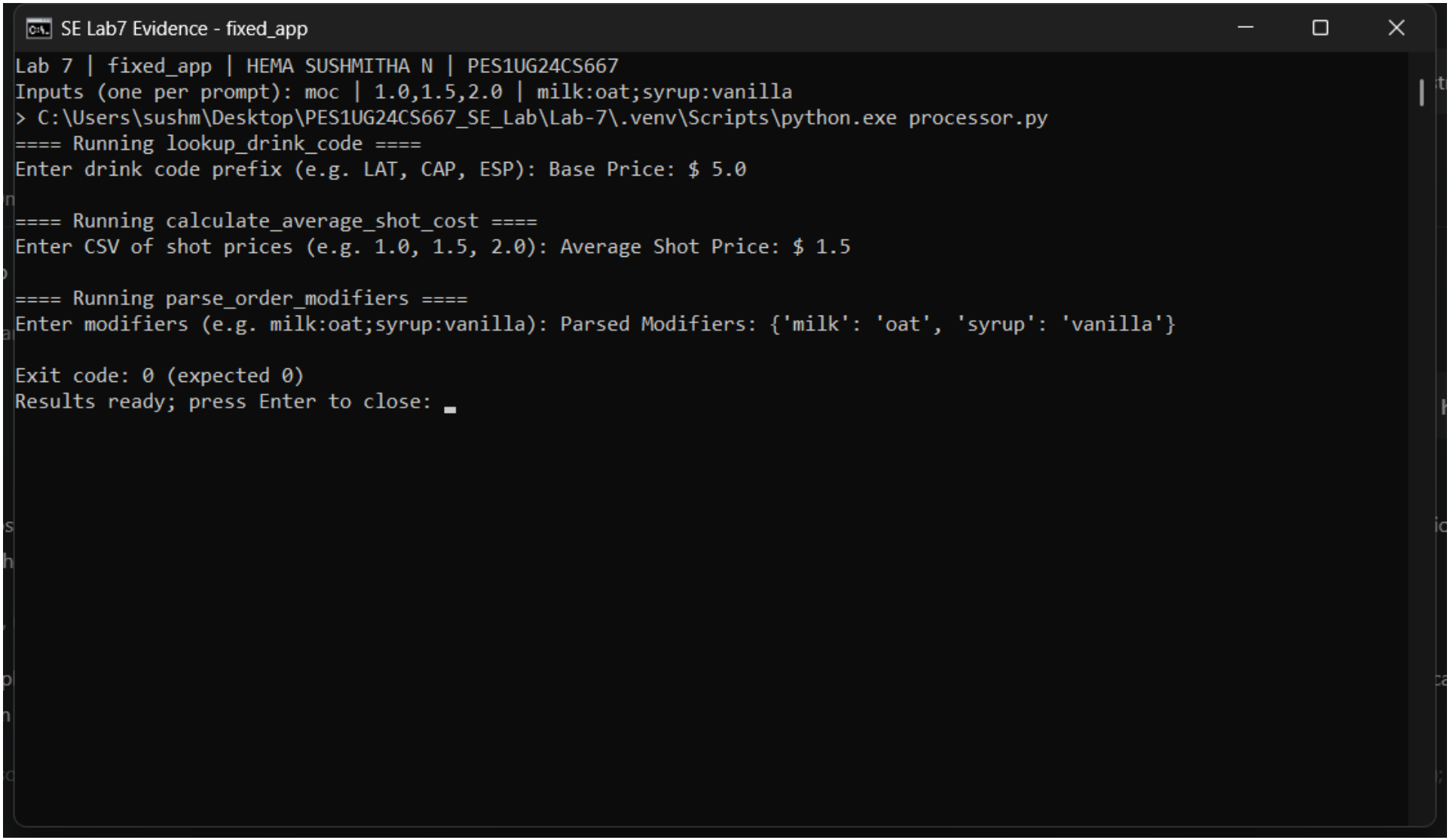
Three completed starter properties against original source | 3 failed, 41 deselected



All three no-crash properties fail. Complete counterexamples and the full 34-failed/10-passed baseline are retained in the raw report.

Step 5: Fixed Application

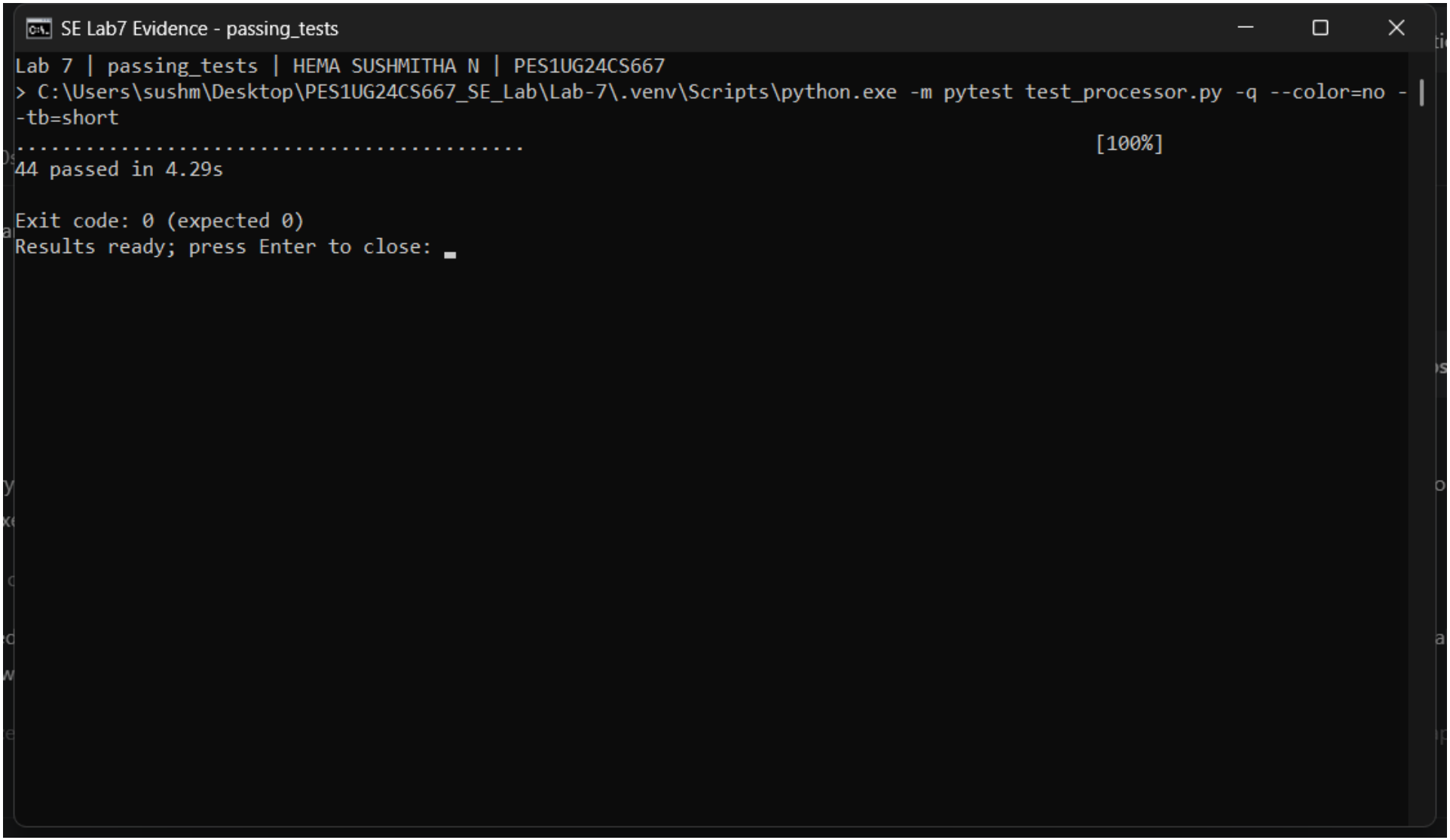
Same demo inputs | price 5.0 | average 1.5 | modifiers parsed | exit 0



The corrected application handles the same inputs and reaches all three processing stages.

Step 6: Passing Tests After Fixes

Identical completed test suite against fixed processor.py | 44 passed



All eight generated properties, 33 regression cases and three CLI tests pass after hardening.